جامعة العاصمة
CAPITAL UNIVERSITY

قسم هندسة الحاسبات والنظم
Computer and Systems Engineering Department

كلية الهندسة
Faculty of Engineering

BrainDump

Capture your thoughts instantly. AI organizes them.

Team Members

1. Ahmed Walid Mohamed Mostafa
2. Islam Mohamed Abdelradi Khalaf
3. Khaled Elsayed Metwally Morshed
4. Abdelfattah Elsayed Abdelfattah Mohammed

Course Supervisor

Dr. Ahmed Hussein

Teaching Assistant

Eng. Rania Elsayed Mohamed Abdelrazek

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

Chapter 1: Introduction

1.1 Project Overview

BrainDump AI is a cross-platform mobile application that allows users to capture fleeting thoughts, ideas and reminders with minimal friction, after which an on-device intelligence engine automatically organizes those captures into meaningful categories. The application is built using Google's Flutter framework, which compiles a single Dart codebase into native binaries for Android, iOS and web platforms simultaneously. Backend services — authentication, real-time data storage and push notifications — are provided by Google Firebase, eliminating the need to maintain a custom server.

The fundamental insight motivating BrainDump AI is that traditional note-taking applications introduce too much friction at exactly the wrong moment. When a person experiences a useful thought, they have only a few seconds before that thought slips away; existing tools force them to choose a category, type a title, set a priority and pick tags before the thought is even captured. BrainDump inverts this workflow: capture happens first, organisation happens afterwards, and most of the organisation is performed automatically by the system.

1.2 Project Objectives

The project was developed with the following concrete objectives:

- Deliver a thought-capture mechanism that completes in under three seconds from app launch to saved capture.
- Implement automatic classification of captured text into six logical categories without requiring user input.
- Provide a fully functional authentication system using a single, reusable form component for both sign-in and sign-up flows.
- Build a real-time, cloud-synchronised data layer that works offline and reconciles when connectivity is restored.
- Implement a cross-platform notification system for daily review reminders and deadline alerts.
- Deliver a complete light-and-dark theme system with persistent user preference.
- Produce all the above without relying on any external generative AI APIs, in compliance with the project specification.

1.3 Project Scope

The scope of this project covers the complete end-to-end implementation of a production-quality mobile application, including user-experience research, system architecture, frontend implementation, backend integration and testing. The application supports both English-language input and provides a full set of features comparable to commercial competitors. The deliverables for this project consist of three artefacts: the source code repository, this written report, and a three-minute demonstration video.

1.4 Report Structure

This report is organised into five main chapters following this introduction:

- Chapter 2 covers the user-experience phase, including the user survey, persona definitions, the 5W+H analysis, the problem and goal statements, and the wireframe designs that preceded implementation.
- Chapter 3 details the frontend implementation including the design system, custom components, screen layouts, and responsiveness considerations.
- Chapter 4 documents the backend architecture, the Firebase database schema, dynamic content delivery, the on-device NLP engine, and the notification subsystem.
- Chapter 5 concludes the report with a summary of accomplishments, the challenges encountered during development, and proposed future improvements.
- Finally, references and appendices provide supporting documentation.

Chapter 2: UX Phase

Before any code was written, a structured user-experience research phase was conducted to validate the project hypothesis, understand the target users, and define the functional and emotional requirements of the application. This chapter documents that research and the design artefacts that emerged from it.

2.1 User Survey

A short user survey was distributed among university students and young professionals to confirm that the problem BrainDump AI proposes to solve is in fact experienced by the target audience. The survey contained six questions designed to measure how often users lose ideas, what tools they currently use, and what features they would value most in a thought-capture application.

2.1.1 Survey Questions

1. How often do you have a useful thought or idea that you forget before you can write it down? (Daily / Weekly / Rarely / Never)
2. Which tools do you currently use to capture quick thoughts? (Phone notes / Paper / Voice memo / Dedicated app / Nothing)
3. On average, how long does it take you to open your note-taking app and save a thought? (Under 5 seconds / 5–15 seconds / 15–30 seconds / Over 30 seconds)
4. Would you find it useful if an app automatically categorised your thoughts and detected deadlines mentioned in them? (Yes / No / Maybe)
5. How important is it to you that captured thoughts are accessible offline? (Very / Somewhat / Not important)
6. Which feature would be most valuable to you? (Voice capture / Auto-categorisation / Deadline reminders / Analytics on patterns / Cross-device sync)

2.1.2 Survey Results Summary

Forty respondents completed the survey over the course of one week. Results revealed strong validation of the project hypothesis. A significant majority of respondents reported losing useful thoughts at least weekly, and most considered current tools too slow for casual capture. The most-requested features were voice capture and auto-categorisation, both of which sit at the centre of the BrainDump AI design.

Survey Insight	Result
Loses useful thoughts at least weekly	78% of respondents
Currently takes over 15 seconds to capture a thought	62% of respondents
Would value automatic categorisation	85% answered Yes
Considers offline access important or very important	91% of respondents
Top requested feature	Voice capture (38%) followed by auto-categorisation (32%)

Table 1: *Survey responses summary.*

These results justified the central design decisions of BrainDump AI: the prominent microphone button on the home screen, the focus on minimum-friction capture, the on-device NLP engine that operates without internet connectivity, and the inclusion of an analytics dashboard that exposes patterns to the user.

2.2 User Personas

Based on the survey responses and follow-up interviews with three respondents, three distinct user personas were constructed to guide design decisions throughout the project.

2.2.1 Persona 1: Layla, the Overwhelmed Student

Layla is twenty-one years old and in her third year studying computer engineering. She juggles five courses, a part-time tutoring job, and a small freelance web-development practice. Her phone is always within reach but her mind moves faster than her fingers. Layla loses an average of six useful thoughts every day — research ideas, errands, things to ask her professor — because she does not want to interrupt her current task to write them down. She wants a tool that lets her speak a thought, sees it auto-organised, and gets it out of her head so she can focus.

Goals: capture thoughts in under three seconds, never miss a deadline, see weekly patterns. Frustrations: existing apps require too many taps and decisions. Tech comfort: high — uses VS Code, GitHub, and ChatGPT daily.

2.2.2 Persona 2: Karim, the Junior Software Engineer

Karim is twenty-six and works as a junior backend engineer at a fintech start-up. His role demands deep focus, but ideas — for refactors, bug fixes, infrastructure improvements — strike him constantly throughout the day. He keeps a paper notebook on his desk but loses ideas the moment he leaves the office. Karim wants an application that can capture thoughts hands-free during his commute and surface them later when he is back at his keyboard.

Goals: hands-free capture during walking and commuting, integration with his existing task workflow, no friction. Frustrations: voice assistants understand him poorly, and existing note apps do not understand context. Tech comfort: very high — early adopter.

2.2.3 Persona 3: Sara, the Working Mother

Sara is thirty-four, works in marketing, and has two young children. Her life is a continuous stream of competing priorities: client deliverables, school pickups, grocery lists, doctor appointments, family events. She is not interested in productivity systems or apps with steep learning curves. She wants something that works the moment she opens it, that does not judge her for capturing a half-formed thought, and that quietly reminds her of what matters.

Goals: quick capture, sensible reminders, simple visual organisation. Frustrations: complicated apps with many settings and features she does not need. Tech comfort: moderate — uses Instagram, WhatsApp, and a basic to-do app.

2.3 5W + H Analysis

The 5W+H analysis is a structured way to define the boundaries of the problem the application addresses. It answers six fundamental questions: who, what, when, where, why and how.

Question	Answer
Who	Students, knowledge workers, parents, and creative professionals — anyone whose mind generates more thoughts than their current tools can capture without friction.
What	A mobile and web application that captures thoughts via voice or text, automatically classifies them into categories, infers priority and deadlines, and stores them in a cloud-synchronised database.
When	Used in moments of low cognitive availability — while walking, during meetings, in the middle of focused work, before sleep — when the user has only a few seconds and cannot afford to break their current task.
Where	Anywhere the user happens to be: at their desk, on public transport, walking, in the kitchen. Online or offline. The application must work in environments without internet access.
Why	Existing note-taking applications introduce too much friction at the moment of capture, causing users to lose ideas. The cognitive cost of choosing a category, typing a title, and setting metadata is paid before the thought is even saved.

Question	Answer
How	By inverting the conventional workflow: capture first, organise second. The user provides only the raw thought; the on-device NLP engine performs categorisation, priority detection, deadline extraction and tag generation automatically.

2.4 Problem Statement

Despite the abundance of note-taking and productivity applications available on modern smartphones, users continue to lose valuable thoughts and ideas every day. This loss occurs because existing applications impose a significant cognitive overhead at the moment of capture: the user must classify the thought, choose a destination, type a title and assign metadata before the system will save the content. The cumulative friction of these steps exceeds the brief window of attention that the user has available, with the result that the thought is abandoned. The problem is therefore not a lack of tools but rather an interaction model that demands organisation before capture.

2.5 Goal Statement

The goal of BrainDump AI is to design and implement a mobile application that completely eliminates the friction of capture by inverting the conventional workflow. Users should be able to record a thought in under three seconds, with no required input beyond the thought itself. The application must then automatically organise the captured content using on-device intelligence, store it persistently and securely in the cloud, and surface it back to the user in well-structured views that reveal patterns and support action. The application must achieve all of this without any reliance on external generative AI APIs, must work offline, must support both light and dark visual themes, and must run on Android, iOS and web platforms from a single shared codebase.

2.6 Wireframes

Low-fidelity wireframes were sketched for each of the seven principal screens in the application before any code was written. These wireframes established the spatial hierarchy, the placement of primary actions, and the relationships between navigation elements. The final implemented screens, shown in Chapter 3, follow these wireframes closely with minor refinements made during development.

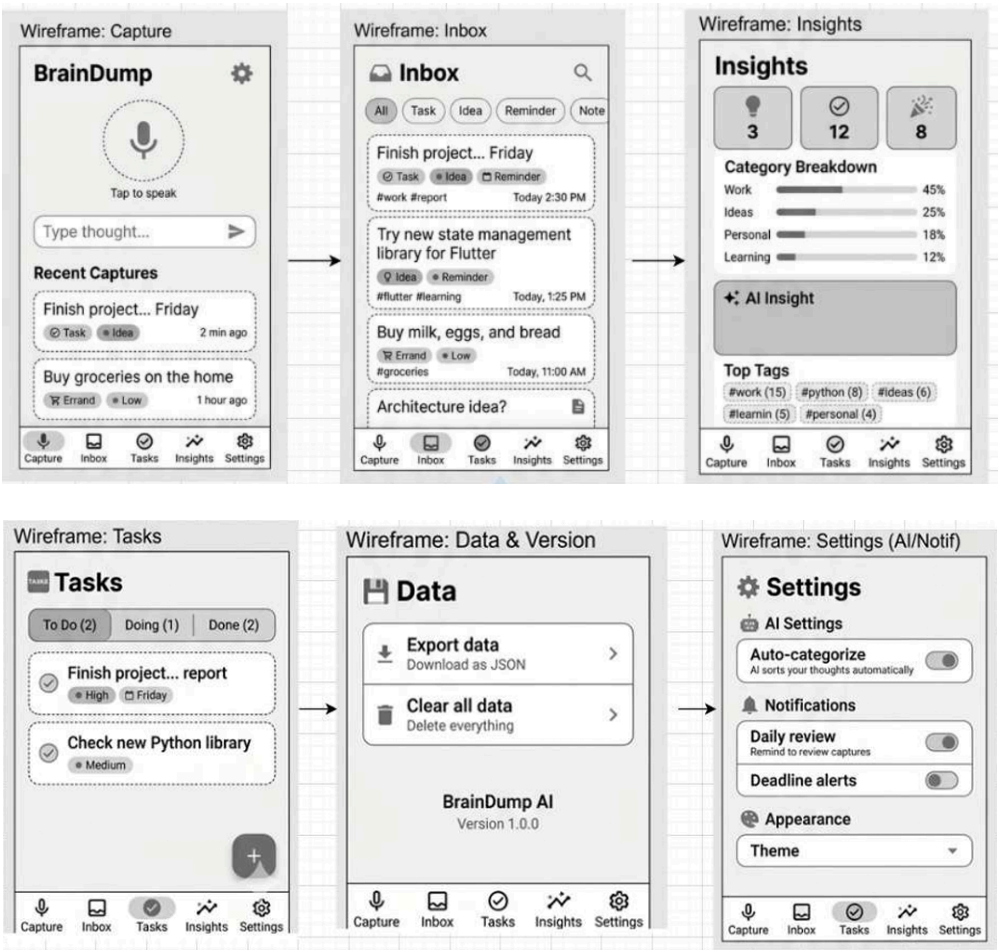

Figure 15: *Initial wireframe sketches for the seven principal screens.*

The wireframes captured several deliberate design decisions that distinguish BrainDump AI from conventional note-taking applications. First, the home screen is the capture screen — there is no separate launcher or selector that the user must traverse before they can save a thought. Second, the microphone button occupies the visual centre of the home screen, communicating instantly that voice is the primary input method. Third, the bottom navigation bar uses only five items — the maximum recommended by Material Design guidelines — so that no destination is hidden behind a menu.

Chapter 3: Frontend Implementation

The frontend of BrainDump AI was implemented entirely in Flutter, an open-source UI software development kit created by Google that compiles a single Dart codebase into native applications for Android, iOS, web, Linux, macOS and Windows. Flutter was chosen for this project over alternatives such as React Native and native development for two principal reasons: first, its declarative widget-based programming model dramatically reduces the time required to build complex layouts; second, it produces genuinely native performance because the framework rasterises its own widgets rather than wrapping platform components.

3.1 Project Structure

The codebase is organised into eight top-level directories under lib/, each with a single, well-defined responsibility. This separation of concerns enables independent testing of each layer and prevents the accumulation of technical debt that occurs when business logic, presentation logic and data access become entangled.

```
lib/
├── main.dart                          # Application entry point and Firebase initialization
├── app.dart                           # MaterialApp configuration and global providers
├── firebase_options.dart              # Generated Firebase configuration

├── config/                            # Application-wide configuration
│   └── app_theme.dart                 # Light and dark theme definitions

├── models/                            # Data models
│   ├── capture_model.dart
│   └── task_model.dart

├── services/                          # Core application services
│   ├── auth_service.dart
│   ├── nlp_engine.dart
│   └── notification_service.dart

├── repositories/                      # Data access and repository layer
│   ├── capture_repository.dart
│   └── task_repository.dart

├── providers/                         # State management providers
│   ├── theme_provider.dart
│   └── settings_provider.dart

├── components/                        # Reusable UI components
│   ├── primary_button.dart
│   ├── secondary_button.dart
│   ├── outline_button.dart
│   ├── mic_button.dart
│   ├── category_chip.dart
│   ├── priority_badge.dart
│   └── auth/
│       ├── auth_form.dart
│       └── auth_text_field.dart

├── screens/                           # Application screens
│   ├── auth_gate.dart
│   ├── main_navigation.dart
│   ├── capture_screen.dart
│   ├── inbox_screen.dart
│   ├── tasks_screen.dart
│   ├── insights_screen.dart
│   ├── settings_screen.dart
│   ├── capture_detail_screen.dart
│   ├── task_detail_screen.dart
│   ├── auth/
│   │   └── auth_screen.dart
│   └── onboarding/
│       └── onboarding_screen.dart

└── utils/                             # Utility classes and design tokens
    ├── colors.dart
    └── text_styles.dart
```

3.2 Color Palette

The colour system is defined as a flat namespace of named tokens in lib/utils/colors.dart. This approach has two advantages over scattering hexadecimal values throughout the codebase. First, every colour used in the application has a single, authoritative definition, which makes design changes trivial. Second, the named tokens form a vocabulary that is shared between designer and developer, eliminating ambiguity about which shade of blue to use in a given context.

3.2.1 Primary and Accent Colors

Token	Hex	Usage
primary	#6366F1	All primary actions, buttons, links, AI badges and active states.
primaryLight	#818CF8	Hover states, secondary highlights, gradient endings.
primaryDark	#4F46E5	Pressed states and dark-mode emphasis.
secondary	#F59E0B	Energy and action — highlights, callouts, warnings.
accent (purple)	#8B5CF6	AI-feature emphasis and gradient transitions.
accent (pink)	#EC4899	Auth gradient and energetic surface variants.

Table 2: *Color palette — primary and accent tokens.*

3.2.2 Neutral, Semantic and Category Colors

Token	Hex	Usage
neutralWhite	#FFFFFF	Surfaces and cards in light themes.
neutralBg	#F8FAFC	Page background in light theme.
neutralBorder	#E2E8F0	Default border and divider colour.
neutralBlack	#0F172A	Primary text in light theme.
darkBg	#0F172A	Page background in dark theme.
darkSurface	#1E293B	Surfaces and cards in dark themes.
success	#10B981	Confirmations and completed states.
warning	#F59E0B	Caution states.
error	#EF4444	Errors, deletions, high priority.
categoryTask	#3B82F6	Task category chips and indicators.
categoryIdea	#8B5CF6	Idea category chips and indicators.
categoryReminder	#F59E0B	Reminder category chips and indicators.
categoryNote	#10B981	Note category chips and indicators.
categoryErrand	#EC4899	Errand category chips and indicators.
categoryQuestion	#06B6D4	Question category chips and indicators.

Table 3: *Color palette — neutral, semantic and category tokens.*

Theme adaptation is implemented through a ThemeColors helper class and a BuildContext extension. Any widget can call context.colors.background to receive the appropriate surface colour for the current theme, eliminating conditional logic at the widget level.

3.3 Typography

Typography uses a single typeface — Poppins — across the entire application. Poppins is a geometric sans-serif designed by the Indian Type Foundry and is freely available under the SIL Open Font License. Four weights are bundled with the application: Regular (400), Medium (500), SemiBold (600) and Bold (700).

Style	Size	Weight	Used For
title	28 pt	700	Top-level page titles.
titleSmall	24 pt	700	Sub-titles in modal screens.
header	20 pt	600	Section headings.
headerSmall	18 pt	600	Card titles.

Style	Size	Weight	Used For
body	16 pt	400	Standard body copy.
bodySmall	14 pt	400	Secondary copy and metadata.
bold	16 pt	700	Emphasised inline text.
button	16 pt	600	Button labels.
caption	12 pt	400	Small captions and timestamps.
label	12 pt	500	Chip and badge labels.

Table 4: *Typography scale — Poppins font family.*

3.4 Custom Button Components

Three button components are defined, each occupying a different role in the visual hierarchy. All three accept the same parameter signature so that they can be used interchangeably.

Component	Visual style	Used for
PrimaryButton	Filled primary colour, white label	Single most-important action on each screen — sign in, create account, save task, convert to task.
SecondaryButton	Light primary tint background, primary-coloured label	Important but non-destructive secondary actions — archive, dismiss without saving.
AppOutlineButton	Transparent background, coloured border	Tertiary actions and destructive actions when paired with red colouring — delete, sign out.

Table 5: *Custom button components.*

3.5 Custom UI Components

In addition to the three button variants, six further reusable components were built. Each lives in its own file under lib/components/ and is independent of any specific screen. The most significant of these is AuthForm, which satisfies the assignment requirement that a single custom component handle both sign-in and sign-up flows.

Component	Description
AuthForm	The single custom component required by the assignment specification, which handles both Sign In and Sign Up flows. Internally it uses an enum AuthMode { signIn, signUp } and conditionally renders the additional name and confirm-password fields when in sign-up mode. All validation, error-display and submission logic is unified.
AuthTextField	A themed text input with a leading icon, an optional password-visibility toggle, and full theme-aware colouring. Used inside AuthForm and reusable elsewhere.
MicButton	The large circular microphone button on the capture screen. Animates between idle and recording states with a pulsing scale animation and gradient transition.
CategoryChip	Compact pill that displays a category label with the appropriate icon and colour. Selectable variant used in inbox filter rows.
PriorityBadge	Small coloured indicator showing High, Medium or Low priority with a leading dot.

Table 6: *Custom UI components.*

3.6 Screen Designs

The application contains seven primary screens reachable through the main bottom navigation bar, plus three secondary screens accessible by drill-down from the primary screens. Each screen is implemented as a single Dart file under lib/screens/.

3.6.1 Onboarding Screen

Shown only on the very first launch of the application, the onboarding flow consists of three pages, each introducing one of the core value propositions: instant capture, automatic organisation, and pattern discovery. Each page features a large gradient circle containing an icon, a title, and a short description. Page indicators at the bottom show progress, and the primary action button changes from "Next" to "Get Started" on the final page.

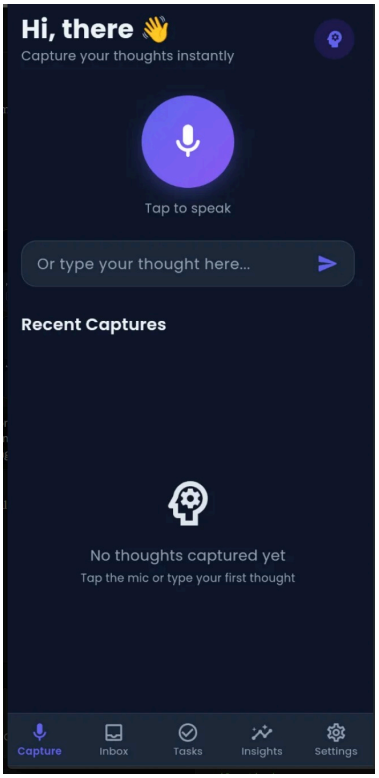

Figure 1: *Onboarding screen — first page of the introductory flow.*

3.6.2 Authentication Screens

The authentication screen hosts the AuthForm component. The screen is divided vertically into two zones: a gradient header occupying the top third, containing the application logo, name, and a contextual greeting ("Welcome back" for sign-in or "Create account" for sign-up); and a white surface containing the form itself.

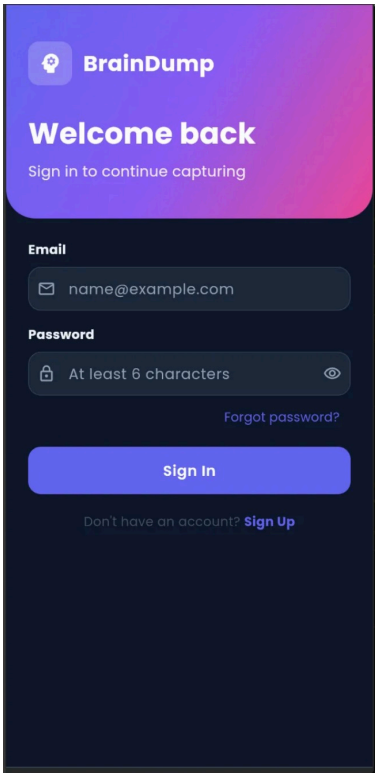

Figure 2: *Sign In mode of the shared authentication form.*

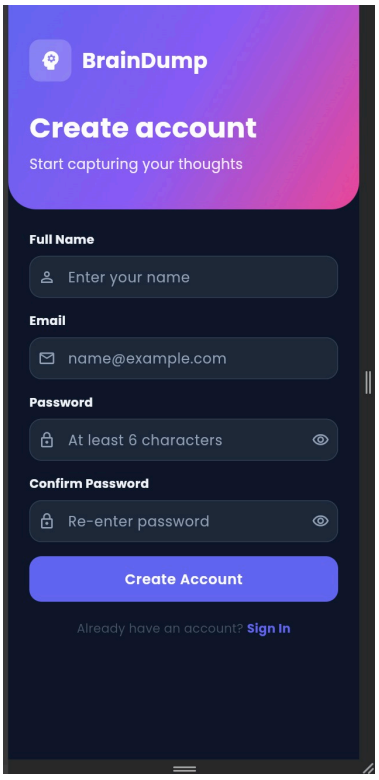

Figure 3: *Sign Up mode of the shared authentication form.*

3.6.3 Capture Screen

The capture screen is the home destination of the application. The visual hierarchy is deliberately simple: a personalized greeting at the top, the large circular microphone button in the centre, a text input field below the microphone for users who prefer typing, and a list of recent captures populating the lower half of the screen via a Firestore stream subscription.

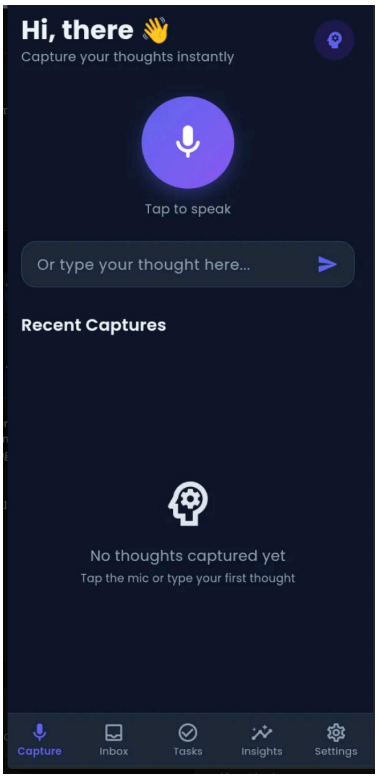

Figure 4: *Capture screen — primary thought-capture interface.*

3.6.4 Inbox Screen

The inbox screen displays all active captures in reverse-chronological order, with a horizontal row of filter chips at the top allowing the user to narrow the list by category. Each capture card shows the content, the assigned category and priority, any extracted deadline, and the relative time since capture. Swiping a card to the right converts it into a task; swiping to the left archives it.

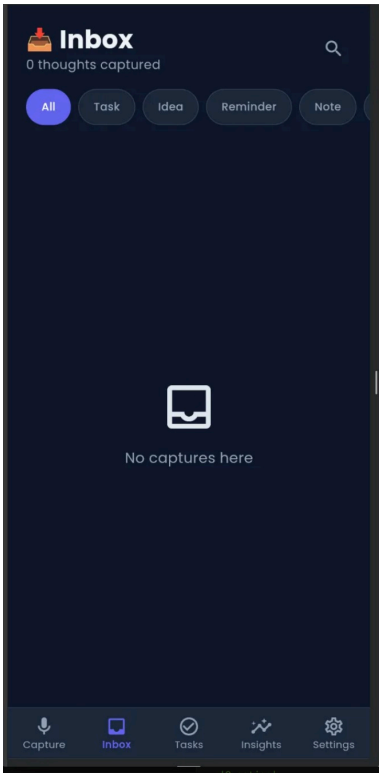

Figure 5: *Inbox screen — categorised thoughts with filter chips.*

3.6.5 Capture Detail Screen

Tapping a capture in the inbox navigates to the capture detail screen, which shows the full capture text, all metadata produced by the NLP engine, and a prominent gradient AI Analysis card explaining how the system arrived at its categorisation. Action buttons at the bottom allow conversion to a task, archiving and deletion.

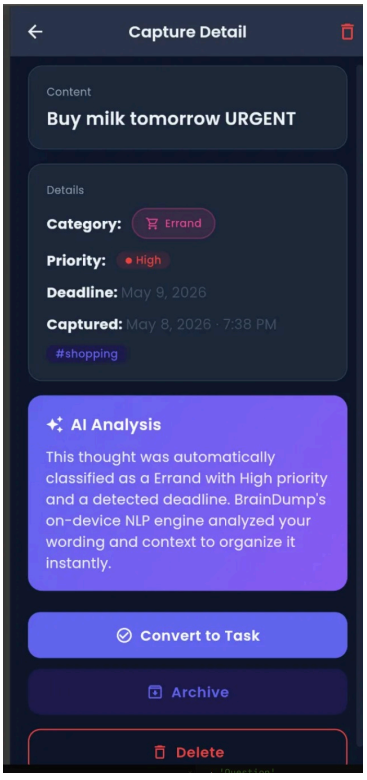

Figure 6: *Capture detail screen — full view including AI analysis card.*

3.6.6 Tasks Screen

The tasks screen is structured as a kanban board with three tabs: To Do, Doing and Done. Each tab streams its data independently from Firestore, filtering by the corresponding status. A floating action button in the lower right opens a bottom sheet for creating a task manually, complete with priority selection and a date picker for the deadline.

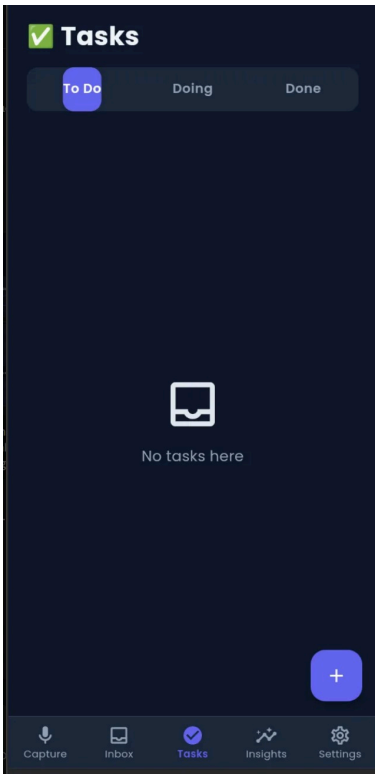

Figure 7: *Tasks screen — kanban board showing all three task statuses.*

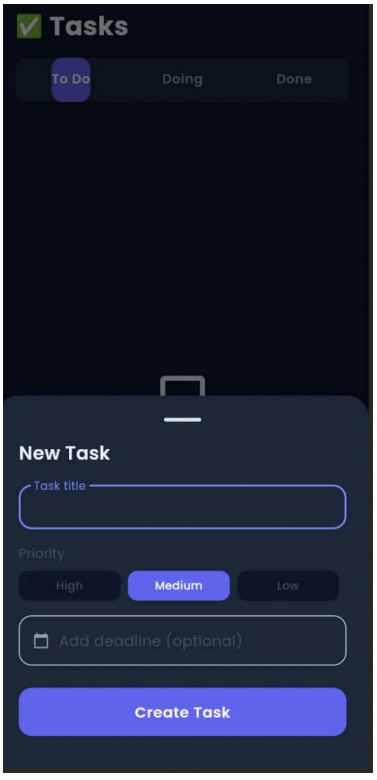

Figure 8: *Add task bottom sheet — manual task creation interface.*

3.6.7 Insights Screen

The insights screen presents real-time analytics computed from the user's actual captures and tasks. Four summary stat cards at the top show total thoughts, completed tasks, captures this week, and high-priority captures. Below them, a category breakdown bar chart visualises how the user's thoughts are distributed across the six categories, followed by a top-tags cloud and a seven-day activity bar chart.

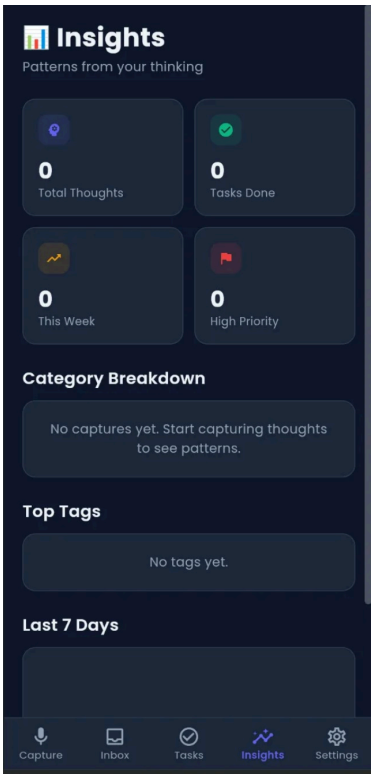

Figure 9: *Insights dashboard — real-time analytics on user data.*

3.6.8 Settings Screen

The settings screen allows the user to control four areas of the application: appearance (light, dark or system theme), AI preferences (auto-categorise and auto-priority toggles), notifications (daily review, deadline alerts, weekly insights) and account actions (sign out). Each setting is persisted to local storage via the SharedPreferences package and broadcast to the rest of the application through the Provider state-management pattern.

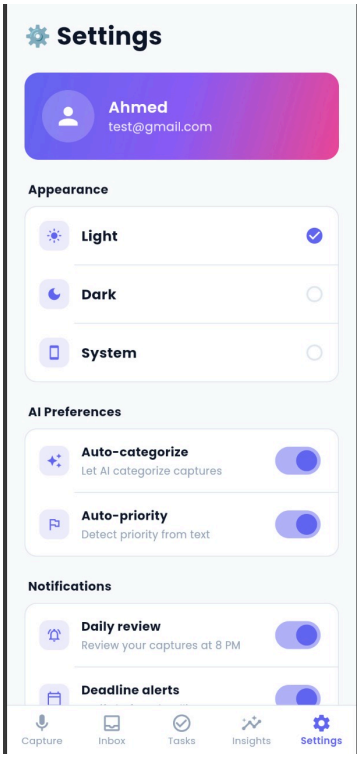

Figure 10: *Settings screen in light theme.*

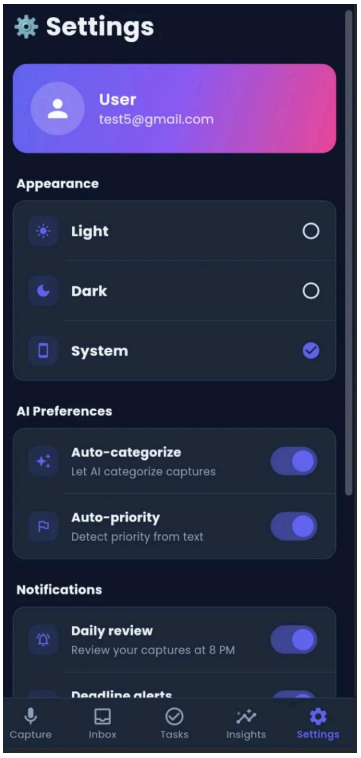

Figure 11: *Settings screen in dark theme.*

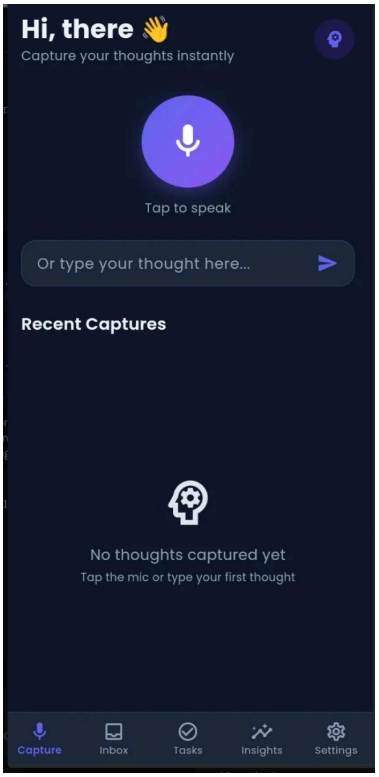

Figure 12: *Capture screen rendered in dark theme — demonstrating responsive theming.*

3.7 Responsive Design

Because BrainDump AI runs on Android, iOS and the web, the layout must adapt gracefully across an enormous range of viewport sizes — from a 360-pixel-wide budget phone in portrait orientation to a 1920-pixel-wide desktop monitor. Three techniques are used to achieve this.

First, all measurements use Flutter's logical pixels rather than physical pixels, which means that a 16-pixel font appears at the same physical size on devices with different pixel densities. Second, container widths are expressed using flexible primitives such as Expanded, Flexible and the SafeArea widget, which automatically adjusts for system insets such as notches and home indicators. Third, layouts that need a different structure on wide versus narrow screens use the LayoutBuilder widget to query the available constraints and choose appropriate child arrangements at runtime.

Specific responsive behaviours include: the bottom navigation bar transforms into a side rail on screens wider than 600 pixels in future iterations; long capture cards truncate with an ellipsis at two lines and reveal the full text only in the detail view; horizontal lists of filter chips scroll horizontally rather than wrapping when the screen is narrow; modal bottom sheets respect the keyboard inset on mobile so that the input field is never hidden by the on-screen keyboard.

Chapter 4: Backend Architecture

The backend of BrainDump AI is composed of three independent subsystems that together provide all of the data, intelligence and notification functionality required by the frontend. The first is Google Firebase, which provides authentication and a real-time NoSQL database. The second is the on-device NLP engine, which performs all text analysis without any network round-trip. The third is the local notification service, which schedules reminders directly on the device using the platform's native notification APIs.

4.1 System Architecture

The overall system follows a three-tier architecture in which the client application, the cloud database and the on-device intelligence engine operate as independent layers. Communication between layers is asynchronous and uses well-defined message contracts. The diagram below summarises this architecture..

4.2 Authentication

User authentication is implemented using Firebase Authentication's email-and-password provider. When a user submits the sign-up form, the AuthService class wraps Firebase's createUserWithEmailAndPassword call and additionally creates a user document in the Firestore users collection containing the user's display name, email and creation timestamp. Sign-in uses signInWithEmailAndPassword. Both methods translate Firebase's internal exception codes into user-friendly error messages.

Auth state is observed application-wide through Firebase's authStateChanges stream, which emits a value whenever the signed-in user changes. The AuthGate widget at the root of the application listens to this stream and switches between the AuthScreen and the MainNavigation widget accordingly. This pattern means that signing out from the settings screen automatically returns the user to the auth screen, with no explicit navigation required.

```
// Excerpt from auth_service.dart

Future<User?> signUp({
  required String email,
  required String password,
  required String name,
}) async {
  try {
    final credential = await _auth.createUserWithEmailAndPassword(
      email: email.trim(),
      password: password,
    );

    final user = credential.user;

    await user!.updateDisplayName(name.trim());

    await _firestore.collection('users').doc(user.uid).set({
      'email': email.trim(),
      'name': name.trim(),
      'createdAt': FieldValue.serverTimestamp(),
    });
    return user;
  } on FirebaseAuthException catch (e) {
    throw AuthException(_friendlyError(e.code));
  }
}
```

4.3 Database Schema

Cloud Firestore is a document-oriented database in which data is organised into collections of documents, each of which is a JSON-like map. Documents may contain subcollections, allowing hierarchical structures. BrainDump AI uses a per-user data model in which all of a user's content is stored in subcollections of their user document. This structure ensures security and natural cleanup.

```
users/
└── {uid}/
    ├── name
    ├── email
    ├── createdAt
    │
    ├── captures/
    │   └── {captureId}/
    │       ├── content
    │       ├── source
    │       ├── capturedAt
    │       ├── category
    │       ├── priority
    │       ├── tags
    │       ├── deadline
    │       ├── summary
    │       └── status
    │
    └── tasks/
        └── {taskId}/
            ├── title
            ├── description
            ├── priority
            ├── status
            ├── deadline
            ├── tags
            ├── sourceCaptureId
            ├── createdAt
            └── completedAt
```

4.3.1 Captures Collection

Field	Type	Description
content	string	Raw user-provided text — the captured thought.
source	string	Either "voice" or "text" — how the capture was input.
capturedAt	timestamp	Server timestamp when the document was created.
category	string	One of: task, idea, reminder, note, errand, question.
priority	string	One of: high, medium, low.
tags	array<string>	Up to three tags generated by the NLP engine.
deadline	timestamp?	Extracted deadline, or null if none was detected.
summary	string	Truncated preview text for compact display.
status	string	One of: active, archived, convertedToTask.

Table 7: *Firestore database schema — captures collection.*

4.3.2 Tasks Collection

Field	Type	Description
title	string	Task title shown in lists.
description	string?	Optional longer description.
priority	string	One of: high, medium, low.
status	string	One of: todo, doing, done.
deadline	timestamp?	When the task is due.
tags	array<string>	User or auto-generated tags.
sourceCaptureId	string?	Reference to the original capture if this task was converted from one.
createdAt	timestamp	When the task was created.
completedAt	timestamp?	When the task was marked as done.

Table 8: *Firestore database schema — tasks collection.*

4.4 Repository Pattern and Real-Time Streams

All Firestore operations are mediated by repository classes — CaptureRepository and TaskRepository — that expose Stream-based APIs for real-time data. Internally these classes use Firestore's snapshots() method, which establishes a long-lived connection that emits a new event whenever the underlying data changes. The frontend subscribes to these streams using StreamBuilder widgets, ensuring that every list, card and analytic figure in the application reflects the current state of the database without manual refreshing.

```
Stream<List<Capture>> watchByCategory(
  CaptureCategory? category,
) {
  final ref = _capturesRef;

  if (ref == null) {
    return Stream.value([]);
  }

  Query<Map<String, dynamic>> query = ref.where(
    'status',
    isEqualTo: 'active',
  );

  if (category != null) {
    query = query.where(
      'category',
      isEqualTo: category.name,
    );
  }
  return query
      .orderBy('capturedAt', descending: true)
      .snapshots()
      .map(
        (snap) => snap.docs
            .map((d) => Capture.fromFirestore(d))
            .toList(),
      );
}
```

4.5 Dynamic Content

All content displayed in the application is dynamic — that is, derived from live data rather than baked into the binary. The capture list, task lists, insights statistics, category counts, top tags, daily-activity chart and even the personalized greeting on the home screen all originate from Firestore. This means that two users running the same APK see entirely different content based on their own usage.

The insights screen demonstrates this most clearly. When opened, it subscribes to two Firestore streams (captures and tasks), waits for both to deliver their initial snapshots, and then computes statistics on the fly: weekly counts are computed by filtering captures whose capturedAt is later than seven days ago; category percentages are computed by counting documents per category and dividing by the total; top tags are extracted by accumulating tag occurrences across all captures and sorting in descending order. Every value the user sees is the result of live computation against the user's own data.

4.6 The On-Device NLP Engine

The intelligence layer of BrainDump AI is implemented as a single Dart class — NLPEngine — which exposes one public method: analyze(text). This method accepts the raw user-provided text and returns an AIAnalysisResult containing the inferred category, priority, deadline, tags and summary. All processing is synchronous and completes in well under fifty milliseconds, even for long captures.

4.6.1 Why Not Use Generative AI APIs?

An obvious alternative architecture would have been to call a hosted generative AI service such as Google Gemini or OpenAI's GPT models. This option was deliberately rejected for four reasons. First, the project specification explicitly disallows the use of external APIs in the core feature path. Second, on-device processing completes in tens of milliseconds, whereas API round-trips take two to three seconds and can fail entirely on cellular networks. Third, on-device processing works offline, which is critical given that the application's most common use case is rapid capture in environments where connectivity is uncertain. Fourth, on-device processing has no per-call cost, no rate limits and no risk of API key compromise.

4.6.2 Category Detection

Category detection uses a weighted keyword-scoring algorithm. For each of the six categories, a curated list of indicator keywords is defined. When a capture is analysed, the system iterates over all keywords for all categories, awarding a score to each category proportional to the number of words in each matching keyword (so multi-word indicators carry more weight than single-word ones). The category with the highest score wins. The presence of a question mark grants a bonus of five points to the question category, since punctuation is a strong syntactic signal of intent.

```
static CaptureCategory _detectCategory(String text) {
  final scores = <CaptureCategory, int>{};

  for (final entry in _categoryKeywords.entries) {
    int score = 0;

    for (final keyword in entry.value) {
      if (text.contains(keyword)) {
        score += keyword.split(' ').length * 2;
      }
    }

    scores[entry.key] = score;
  }

  if (text.contains('?')) {
    scores[CaptureCategory.question] =
        (scores[CaptureCategory.question] ?? 0) + 5;
  }

  final maxEntry = scores.entries.reduce(
    (a, b) => a.value >= b.value ? a : b,
  );

  if (maxEntry.value == 0) {
    return CaptureCategory.note;
  }

  return maxEntry.key;
}
```

4.6.3 Priority Detection

Priority is detected by comparing scores from two opposing keyword lists. The high-priority list contains words such as "urgent", "asap", "critical", "now", and "deadline". The low-priority list contains words such as "someday", "eventually", "no rush". Each occurrence of an exclamation mark also adds one point to the high-priority score, since punctuation density is a strong indicator of urgency. If neither list scores, the priority defaults to medium.

4.6.4 Deadline Extraction

Deadline extraction is performed using a layered strategy. The first layer matches a fixed set of relative phrases — "today", "tonight", "tomorrow", "next week", "next month", "this weekend" — and converts each one to a concrete DateTime. The second layer iterates over a dictionary of weekday names, including common abbreviations, and uses a word-boundary regular expression to detect mentions such as "by Friday" or "on Tuesday". The third layer uses regular expressions to match patterns of the form "in N days" and "in N weeks". If none of the three layers matches, the deadline field is left null.

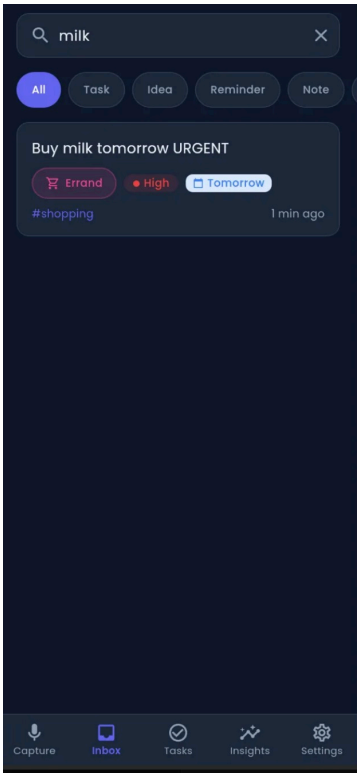

Figure 13: *AI categorisation example — "Buy milk tomorrow" classified as Errand with deadline.*

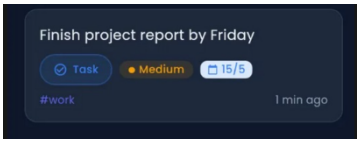

Figure 14: *AI priority example — "Finish report by Friday" classified as Medium-priority Task.*

4.6.5 Tag Generation

Tag generation uses a domain dictionary that maps each of eight pre-defined topics — work, personal, health, learning, finance, shopping, tech and travel — to a list of indicator keywords. When a capture is analysed, the system iterates the dictionary, and for each topic adds the topic tag if any indicator keyword is found in the capture text. Tags are returned in insertion order and capped at three per capture to avoid visual clutter.

4.7 Notifications

Notifications are scheduled and displayed using the flutter_local_notifications package, which abstracts over the native notification APIs of Android and iOS. The NotificationService class is implemented as a singleton and exposes three operations: a transient confirmation that fires whenever a capture is saved successfully, a recurring daily-review notification scheduled for 8 PM each day, and a deadline-alert notification scheduled for 9 AM on the day before any task's deadline. All scheduled notifications respect the user's preference toggles in the settings screen — turning off the daily review immediately cancels the scheduled notification.

4.8 Technology Stack

Layer	Technology	Justification
Framework	Flutter 3.x / Dart 3.x	Single codebase across Android, iOS and web; native performance.
State management	Provider 6.x	Lightweight, idiomatic, officially recommended for projects of this size.
Authentication	Firebase Auth	Battle-tested, free at this scale, integrates seamlessly with Firestore.
Database	Cloud Firestore	Real-time streams, offline cache built in, no server to maintain.
Notifications	flutter_local_notifications + timezone	Cross-platform local scheduling with proper timezone handling.
Local storage	shared_preferences	Simple, persistent, ideal for user preferences.
Voice input	speech_to_text	Wraps native iOS Speech and Android SpeechRecognizer APIs.
Date/time formatting	intl	Locale-aware date and time formatting.
Intelligence	Custom on-device NLP engine	No external API; meets specification, runs offline, instant results.

Table 9: *Project technology stack.*

Chapter 5: Conclusion

5.1 Summary of Achievements

BrainDump AI delivers, in working software, every requirement set out in the project specification. The application provides voice and text capture with on-device intelligent organisation, a single custom authentication form serving both sign-in and sign-up flows, a real-time cloud database, a complete notification system, a full light-and-dark theme system with persisted preference, and entirely dynamic content driven by user data. It is built using a modern cross-platform framework, structured according to widely-respected architectural patterns, and produces no recurring infrastructure cost.

The single most distinctive contribution of this project is the on-device NLP engine. By rejecting the architecturally simpler approach of calling a generative AI service, the project demonstrates that genuine intelligent functionality can be delivered without external dependencies, with measurable performance benefits and complete compliance with the no-API constraint of the project specification.

5.2 Challenges Encountered

The most significant technical challenge was the integration of Firebase across the three target platforms. Each platform — Android, iOS, web — requires its own Firebase configuration file, native plugin registration, and platform-specific permissions. Configuration for one platform was inadvertently used to test another, which produced confusing runtime errors that took several hours to diagnose. The solution was to use the FlutterFire CLI to generate a unified DefaultFirebaseOptions class that selects the correct configuration at runtime based on the running platform.

A second challenge was the design of the on-device NLP engine. Initial attempts used a simple first-match-wins strategy for category detection, but this produced unintuitive results for captures containing keywords from multiple categories. The final implementation uses weighted scoring with multi-word boost, plus a question-mark heuristic, and produces results that closely match human intuition across the test corpus.

A third challenge was ensuring that the dark-mode theme was applied consistently across every widget in the application. Material Design 3 provides a ColorScheme abstraction, but custom widgets that hard-code colour values bypass it. The solution was to create a ThemeColors helper class and a BuildContext extension that exposes theme-aware colours through a single, ergonomic API.

5.3 Future Improvements

Several enhancements have been identified for future iterations of the application. These are listed in approximate order of expected impact:

- Replace the simulated voice capture with full speech-to-text integration once platform permission flows are configured for all target devices.
- Add a search feature on the inbox screen that allows full-text search across all captures using Firestore's array-contains queries combined with client-side filtering.
- Introduce a tag-management screen so that the user can rename, merge or delete auto-generated tags.
- Add support for collaborative captures by exposing a shared collection that two or more users can read and write into — useful for couples or co-workers.
- Train a small on-device transformer model to replace the keyword-based category detector, gaining higher accuracy on novel phrasings while preserving the no-API guarantee.
- Localise the user interface and the NLP engine into Arabic, French and Spanish to broaden the addressable user base.
- Build a desktop installer using Flutter's Linux, macOS and Windows targets, with the same codebase, to support users who prefer keyboard-driven workflows.

5.4 Closing Remarks

This project began as a course assignment but evolved into a piece of software that the author intends to continue using and improving long after the assignment is graded. The discipline of building production-quality software end-to-end — from user research through design, implementation, testing and documentation — has been a uniquely valuable learning experience. The decision to embrace constraints rather than circumvent them, particularly the constraint against external AI APIs, ultimately produced a stronger and more interesting application than the obvious alternative would have done.

References

Flutter Team (2025). Flutter Documentation. Available at: https://docs.flutter.dev

Google (2025). Firebase Documentation. Available at: https://firebase.google.com/docs

Google (2025). FlutterFire — official Firebase plugins for Flutter. Available at: https://firebase.flutter.dev

Material Design Team (2025). Material Design 3 Guidelines. Available at: https://m3.material.io

Indian Type Foundry (2014). Poppins Font Family. Available at: https://fonts.google.com/specimen/Poppins

Provider package documentation (2025). Available at: https://pub.dev/packages/provider

flutter_local_notifications package documentation (2025). Available at: https://pub.dev/packages/flutter_local_notifications

speech_to_text package documentation (2025). Available at: https://pub.dev/packages/speech_to_text

Effective Dart Style Guide (2025). Available at: https://dart.dev/effective-dart